ProsperoEden

محاكي Nintendo Switch على PlayStation 5

دليل كامل لما قمنا به فعليًا: التثبيت، المسارات، الأخطاء، الحلول، والمحاولات التي نجحت والتي فشلت

Firmware	13.40
إصدار المحاكي في جلستنا	v1.000.070
Title ID	PPSA99008
النتيجة النهائية	prod.keys detected + 238 NCA + 1 game detected

ملاحظة: هذا الملف يوثق ما حصل في جلستنا فعلًا، وليس مجرد نسخ لتعليمات README. عند اختلاف المسار الرسمي عن الحل الذي نجح معنا، تم توضيح ذلك صراحة.

1) النتيجة النهائية باختصار

وصلنا إلى مرحلة صار فيها ProsperoEden قادرًا على قراءة ملفات الـSwitch التي تم استخراجها من جهازك وألعابك، بعد سلسلة محاولات بسبب مشكلة وصول المحاكي إلى مسارات الـUSB من داخل متصفح الملفات الخاص به.

النتيجة	الخطوة / المشكلة	الحالة
المحاكي اشتغل على Firmware 13.40.	تشغيل ProsperoEden على PS5	نجح
ظهرت نتيجة prod.keys found.	التعرف على المفاتيح	نجح
تم اكتشاف 238 ملف NCA.	التعرف على الـFirmware	نجح
تم اكتشاف لعبة واحدة في المكتبة.	التعرف على لعبة	نجح
المحاكي اشتغل لكن الأداء كان بطيئًا/متذبذبًا حسب اللعبة؛ بقيت مرحلة تحسين الأداء تجريبية.	الأداء	جزئي

2) البيئة التي كنا نعمل عليها

- PS5 على Firmware 13.40 مع Jailbreak شغال.
- ELF Loader يستمع على المنفذ 9021 أثناء جلسة الجيلبريك.
- تم استخدام kstuff وetaHEN وShadowMountPlus ضمن بيئة الهومبرو.
- الهارد الخارجي ظهر للنظام تحت /mnt/usb0.
- المحاكي المستخدم: ProsperoEden، ومعرّف التطبيق PPSA99008.
- النسخة المستخدمة في جلستنا: v1.000.070.
- الـkeys والـfirmware والألعاب كانت مستخرجة من العتاد والبرامج المملوكة للمستخدم.

3) تجهيز الملفات على الكمبيوتر قبل النقل

في البداية جهزنا مجلد بيانات للمحاكي على الهارد. الفكرة كانت فصل المفاتيح والـFirmware والألعاب والتحديثات في مجلدات واضحة.

الشكل الذي جهزناه لبيانات المحاكي:

```
Eden/
├── keys/
│   ├── prod.keys
│   └── title.keys        (اختياري)
├── firmware/
│   └── *.nca
├── roms/
│   ├── *.xci
│   └── *.nsp
├── updates/
├── mods/
├── save-import/
├── ryujinx/
└── save-export/
```

نقطة مهمة جدًا ظهرت أثناء العمل: ملفات الـFirmware يجب أن تكون ملفات NCA فعلية داخل مجلد firmware، وليس ملف ZIP مغلق. أما الألعاب فتبقى بصيغها مثل XCI أو NSP.

ProsperoEden 4) تجهيز وتشغيل الهومبرو ثم تركيب

- تم تنفيذ الجيلبريك بنجاح حتى أصبح الـELF loader متاحًا على المنفذ 9021.
- تم تشغيل/إرسال المكونات المطلوبة مثل kstuff وShadowMountPlus وetaHEN ضمن جلسة الجيلبريك.
- ShadowMountPlus تعرّف على مجلد التطبيق PPSA99008 الموجود على الـUSB وقام بتركيبه بحيث يظهر كتطبيق PS5.
- المسار الذي استخدمناه على الـUSB كان مرتبطًا بـ /mnt/usb0/PPSA99008.

مهم: الـREADME الحديث للمشروع يوصي غالبًا بوضع التطبيق نفسه في /data/homebrew/PPSA99008 ثم اختيار مجلد بيانات مستقل. لكن جلستنا الفعلية اعتمدت على تركيب التطبيق من الـUSB بواسطة ShadowMountPlus.

5) أول مشكلة: متصفح ProsperoEden لا يرى /mnt/usb0

المشكلة الأساسية كانت أن الهارد ظاهر للنظام وWeb File Manager يراه، لكن متصفح المجلدات داخل ProsperoEden لم يعرض /mnt/ بالشكل المتوقع. لذلك لم نستطع ببساطة اختيار مجلد Eden الموجود على الهارد.

النتيجة	الخطوة / المشكلة	الحالة
متصفح ProsperoEden لم يعرض مسار /mnt/ بالشكل الذي نحتاجه.	اختيار /mnt/usb0 مباشرة من داخل المحاكي	فشل
ShadowMount والنظام كانا يرَيان /mnt/usb0، فالمشكلة كانت في الوصول من داخل التطبيق لا في الهارد.	الهارد نفسه كان سليمًا	معلومة

6) المحاولة الأولى للحل: نقل بيانات Eden إلى /data/Eden

بدل نقل كل شيء عشوائيًا، جرّبنا جعل بيانات Eden في مساحة يمكن للمحاكي الوصول لها بسهولة أكبر.

المسار الذي أنشأناه/استخدمناه:

```
/data/Eden
```

من داخل المحاكي ظهر المسار بصورة قريبة من /app0/Eden. هذه كانت علامة غريبة لكنها مفيدة لأنها أثبتت أن التطبيق يتعامل مع المسارات داخل بيئة sandbox/mount مختلفة عن الشكل الذي نراه من Web File Manager.

في هذه المرحلة ظهرت نتيجة قوية جدًا: المحاكي استطاع عدّ البيانات وأظهر:

- prod.keys found
- NCA files 238
- game 1
- Next launch: /app0/Eden

لكن بقي سلوك متعلق بمسار /app0/assets، لذلك لم نعتبر هذه المحاولة هي الحل النهائي الأنظف.

7) الارتجال الذي نجح فعليًا: استخدام **assets** داخل مجلد التطبيق

هذه أهم نقطة في الجلسة، وهي الحل العملي الذي أنهى مشكلة missing keys عندنا. بدل الاستمرار في إجبار متصفح المحاكي على رؤية /mnt، تم الاستفادة من مسار التوافق القديم الذي كان ProsperoEden يقرأ منه في إصدارات alpha.

تم إنشاء مجلد **assets** داخل مجلد التطبيق المركب على الـ**USB**، ثم وضع بيانات **Eden** داخله.

الشكل العملي الذي نجح معنا:

```
/mnt/usb0/PPSA99008/
└── assets/
    ├── keys/
    │   └── prod.keys
    ├── firmware/
    │   └── *.nca
    ├── roms/
    │   ├── *.xci
    │   └── *.nsp
    └── updates/
        └── *.nsp
```

بعدها صار المحتوى ظاهرًا للمحاكي من داخل التطبيق كمسار /app0/assets، واختفت رسالة فقدان المفاتيح.

8) لماذا نجح حل **assets**؟

بعد مراجعة توثيق المشروع لاحقًا، اتضح أن إصدارات ProsperoEden القديمة كانت أصلًا تقرأ من /data/homebrew/PPSA99008/assets/. التوثيق الحديث يقول إن هذا المسار القديم ما زال مدعومًا للتوافق حتى يتم اختيار مجلد Game files جديد. هذا يفسر لماذا الحل الذي اكتشفناه/طبقناه نجح بدل الاستمرار مع متصفح المجلدات.

بمعنى آخر: لم يكن تغيير اسم عشوائيًا، بل رجعنا عمليًا لمسار توافق قديم يعرفه التطبيق مسبقًا.

9) النتيجة بعد الحل النهائي

النتيجة	الخطوة / المشكلة	الحالة
المحاكي أعلن أن المفتاح موجود.	prod.keys	نجح
تم اكتشاف 238 ملف NCA.	Firmware	نجح
تم اكتشاف لعبة واحدة.	Library	نجح
رسالة فقدان المفاتيح اختفت بعد ترتيب assets.	Missing keys	نجح
العمل الوظيفي نجح، لكن الأداء لم يكن مثاليًا وكان يحتاج ضبطًا إضافيًا حسب اللعبة.	الأداء داخل اللعبة	جزئي

10) المشاكل التي واجهتنا والحلول التي جربناها

النتيجة	الخطوة / المشكلة	الحالة
mnt/ لم يكن ظاهرًا داخل المتصفح رغم أن النظام نفسه يرى /mnt/usb0.	الاعتماد على أن متصفح المحاكي سيعرض USB مباشرة	فشل
المحاكي بدأ يرى البيانات وأظهر /app0/Eden ونتائج صحيحة، لكن بقيت مشكلة مسار التوافق/assets.	استخدام /data/Eden	جزئي
المحاكي قرأ المفاتيح والـFirmware واللعبة بنجاح.	إنشاء /mnt/usb0/PPSA99008/assets	نجح
صار عدّ ملفات الـFirmware صحيحًا:	فك Firmware إلى ملفات NCA	نجح

238 ملفًا.		
لم نحتج لتحويل الألعاب لصيغة أخرى حتى تظهر في المكتبة.	إبقاء الألعاب بصيغة XCI/NSP	معلومة

11) الارتجالات التي تمت في الجلسة نفسها

هذه ليست خطوات README محفوظة؛ هذه الأشياء ظهرت أثناء التجربة الفعلية:

- بدل الاستمرار مع متصفح الملفات الذي لا يظهر /mnt، تم تجربة مسار داخلي data/Eden/ للوصول إلى البيانات من داخل التطبيق.
- تم ملاحظة أن المحاكي يعرض المسار داخليًا كـ app0/Eden/، وهذا أعطانا clue أن التطبيق يعمل عبر mount/sandbox وليس بنفس المسارات الخام.
- تم إنشاء assets داخل PPSA99008 على الـUSB واستخدامه كمسار توافق قديم، وهو الذي أنهى مشكلة missing keys عمليًا.
- تم الاعتماد على ShadowMountPlus لتركيب التطبيق من الـUSB بدل نقل التطبيق بالطريقة التقليدية إلى التخزين الداخلي في تلك الجلسة.
- تم فحص النتيجة بالأرقام بدل الاكتفاء بأن التطبيق يفتح: 238 NCA، prod.keys، و1 game.

12) الأداء بعد نجاح الإعداد

بعد أن صار المحاكي يقرأ البيانات، ظهرت ملاحظة أن الأداء كان فيه lag. ناقشنا خيارات أداء مثل استخدام Vulkan وخيارات تقلل التقطيع/تسرّع تجميع الشيدر، لكن لم نعتبر كل هذه التعديلات إعدادًا نهائيًا ثابتًا لأن ProsperoEden تجريبي والتوافق يختلف من لعبة إلى لعبة.

التوثيق الحالي للمشروع يؤكد أن هناك مفاتيح متعددة توازن بين الدقة والسرعة، لذلك الأفضل تعديل الأداء لعبة بلعبة وعدم تشغيل كل خيارات السرعة دفعة واحدة.

13) نسخة مختصرة لإعادة نفس الإعداد الذي نجح معنا

إذا أردنا إعادة نفس جلسة النجاح حرفيًا، فهذا هو التسلسل المختصر:

1. شغّل الجيلبريك وتأكد أن ELF loader شغال، ثم شغّل kstuff وShadowMountPlus وetaHEN حسب بيئتك المعتادة.
2. ضع مجلد التطبيق PPSA99008 على الـUSB بالطريقة التي يستخدمها ShadowMountPlus عندك.
3. داخل PPSA99008 أنشئ مجلد assets.
4. داخل assets أنشئ keys وfirmware وroms، وupdates إذا احتجته.
5. ضع prod.keys داخل assets/keys.
6. ضع ملفات الـFirmware المفكوكة بصيغة NCA داخل assets/firmware.
7. ضع الألعاب بصيغة XCI/NSP داخل assets/roms.
8. شغّل ShadowMountPlus حتى يظهر ProsperoEden على واجهة PS5.
9. افتح ProsperoEden، ثم أغلقه وافتحه مرة ثانية إذا غيرت المسار أو الملفات، لأن فحص الإعداد يتم عند فتح التطبيق.
10. تحقق من العدادات بدل التخمين: المفاتيح موجودة، ملفات الـFirmware ظاهرة، وعدد الألعاب صحيح.

14) الفرق بين الطريقة التي نجحت معنا والطريقة الرسمية الحديثة

النتيجة	الخطوة / المشكلة	الحالة
التطبيق في /data/homebrew/PPSA99008، وبيانات الألعاب في أي Game files folder يمكن للمحاكي قراءته مثل /mnt/usb0/eden/.	الطريقة الحديثة	معلومة
التطبيق مركب من USB عبر ShadowMountPlus، واستعملنا PPSA99008/assets كمسار توافق قديم	طريقتنا في الجلسة	معلومة

لأنه كان ظاهرًا للمحاكي.		
نسختنا/بيئتنا وقتها لم تعرض /mnt داخل متصفح ProsperoEden، فاضطررنا لمسار توافق موجود أصلًا في المشروع.	لماذا الاختلاف؟	معلومة

15) ملفات ومسارات مهمة للرجوع لها

```
App Title ID: PPSA99008
USB mount: /mnt/usb0
Session app path: /mnt/usb0/PPSA99008
Working compatibility path: /app0/assets
Working USB layout: /mnt/usb0/PPSA99008/assets
Modern app path (README): /data/homebrew/PPSA99008
Modern data path example: /mnt/usb0/eden
Modern app data: /data/prosperoeden
```

16) المرجع الرسمي الذي يفسر الحل

توثيق ProsperoEden الحالي يذكر أن الإصدارات القديمة كانت تقرأ من مجلد assets داخل PPSA99008، وأن هذا المسار يبقى مدعومًا للتوافق إلى أن يتم اختيار Game files folder جديد. هذا يطابق الحل الذي نجح معنا عمليًا.

المستودع المرجعي:

```
https://github.com/A1vinSmith/prosperoeden
```

17) الخلاصة

أكبر مشكلة لم تكن في **prod.keys** ولا في ملفات الـ**Firmware** نفسها، بل في وصول التطبيق إلى المسار. بمجرد أن استخدمنا مسارًا يستطيع **ProsperoEden** رؤيته فعليًا - ثم رجعنا لمسار **assets** القديم داخل التطبيق - اشتغل الاكتشاف: المفتاح ظهر، 238 ملف **NCA** انقرأ، ولعبة واحدة ظهرت في المكتبة. هذا هو الجزء الأهم الذي يستحق الاحتفاظ به لو احتجنا نعيد التجربة لاحقًا.